Manual Técnico - Analizador Markdown

1. Arquitectura

El sistema utiliza una arquitectura por fases:

```
Archivo Markdown
      |
      v
Lexer: caracteres -> tokens
      |
      v
Parser: tokens -> AST
      |
      v
Semántico: AST -> errores semánticos
      |
      v
Controlador
      |
      +--> Reporte HTML renderizado
      +--> Reporte de estadísticas
      +--> Tabla de contenido
```

2. Punto de entrada

`Proyecto2/app.py` ofrece dos modos:

```
python .\Proyecto2\app.py
python .\Proyecto2\app.py archivo.md --consola
```

La primera forma inicia Tkinter. La opción `--consola` ejecuta el pipeline sin interfaz gráfica.

3. Analizador léxico

`lexer.py` implementa un recorrido manual carácter por carácter.

El puntero principal es `self.posicion`. Los métodos centrales son:

- `actual()`: consulta el carácter actual.
- `siguiente()`: consulta un carácter posterior.
- `avanzar()`: consume un carácter y actualiza línea y columna.
- `retroceder()`: deshace un avance tentativo.
- `siguiente_token()`: selecciona la regla léxica aplicable.
- `analizar()`: produce la lista completa de tokens.

Tokens principales:

```
NUMERAL, VIÑETA, INDENT, DEDENT, CITA,
CERCA_CODIGO, REGLA_HORIZONTAL, NEGRITA,
CURSIVA, CODIGO_INLINE, TEXTO,
SALTO_PARRAFO, EOF
```

Los tokens incluyen número, tipo, lexema, línea y columna.

Los bloques de código se tratan como texto literal entre delimitadores de tres backticks. El contenido interno no se tokeniza como negrita, cursiva o viñeta.

4. Analizador sintáctico

`parser.py` implementa un parser descendente recursivo. Mantiene un puntero sobre la lista de tokens, independiente del puntero del lexer.

Métodos principales:

- `mirar(tipo)`: observa el token actual.
- `avanzar()`: consume el token actual.
- `esperar(tipo)`: consume un token obligatorio o genera `ErrorSintactico`.
- `documento()`: reconoce la secuencia de bloques.
- `encabezado()`, `parrafo()`, `lista()`, `cita()` y `bloque_codigo()`.
- `enfasis()`: procesa negrita, cursiva y código inline.

El resultado es un AST representado con diccionarios y listas. Ejemplo:

```
{
    "tipo": "ENCABEZADO",
    "nivel": 2,
    "contenido": [
        {"tipo": "TEXTO", "valor": "Título"}
    ]
}
```

5. Análisis semántico

`semantic.py` recibe el AST, no los caracteres originales ni el puntero del lexer.

Valida:

- Documento y bloques conocidos.
- Niveles de encabezado entre 1 y 6.
- Contenido no vacío.
- Listas con elementos `ITEM`.
- Listas anidadas válidas.
- Niveles de citas mayores que cero.

- Bloques de código con contenido textual.
- Reglas horizontales con guiones.
- Tipos inline permitidos.

6. Controlador

`controlador.py` contiene `ControladorMarkdown`, que coordina las fases:

```
controlador = ControladorMarkdown()
resultado = controlador.analizar_archivo("documento.md")
```

El resultado contiene:

- tokens;
- AST;
- errores léxicos;
- error sintáctico;
- errores semánticos;
- advertencias;
- estadísticas.

Si el documento tiene errores, el flujo conserva la información y puede generar un HTML de diagnóstico.

7. Generación de reportes

`GeneradorHTML` transforma los nodos del AST:

Nodo	HTML
`ENCABEZADO`	`<h1>` a `<h6>`
`PARRAFO`	`<p>`
`LISTA`	`<ul>` y `<li>`
`CITA`	`<blockquote>`
`BLOQUE_CODIGO`	`<pre><code>`
`REGLA_HORIZONTAL`	`<hr>`
`NEGRITA`	`<strong>`
`CURSIVA`	`<em>`
`CODIGO_INLINE`	`<code>`

`guardar_reportes()` genera:

```
documento_renderizado.html
estadisticas.html
```

```
tabla_contenido.html
```

La tabla de contenido usa identificadores como `encabezado-1` para enlazar cada entrada con su encabezado.

8. Interfaz gráfica

`vista.py` utiliza Tkinter y solo se encarga de la presentación:

- selección de archivos;
- ejecución del controlador;
- visualización de estado y estadísticas;
- selección de carpeta de salida;
- notificación de reportes generados.

La lógica del análisis permanece en el controlador y sus módulos especializados.

9. Complejidad y limitaciones

El lexer realiza un recorrido lineal aproximado sobre la entrada. El parser recorre los tokens y el semántico recorre el AST.

El proyecto implementa el subconjunto Markdown definido para la práctica. No pretende cubrir todas las reglas de CommonMark, como tablas, imágenes, HTML incrustado o enlaces avanzados.

En presencia de un error sintáctico, el parser informa el primer punto detectado. El controlador continúa y puede producir un HTML de diagnóstico con el texto original y la lista de errores.